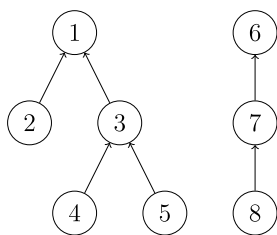


Dsu

并查集



引入

并查集是一种用于管理元素所属集合的数据结构，实现为一个森林，其中每棵树表示一个集合，树中的节点表示对应集合中的元素。

顾名思义，并查集支持两种操作：

- 合并（Union）：合并两个元素所属集合（合并对应的树）
- 查询（Find）：查询某个元素所属集合（查询对应的树的根节点），这可以用于判断两个元素是否属于同一集合

并查集在经过修改后可以支持单个元素的删除、移动；使用动态开点线段树还可以实现可持久化并查集。

► Warning

初始化

初始时，每个元素都位于一个单独的集合，表示为一棵只有根节点的树。方便起见，我们将根节点的父亲设为自己。

▼ 实现



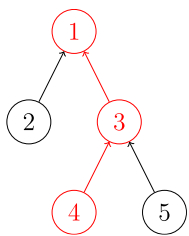
C++Python

```
1 struct dsu {
2     vector<size_t> pa;
3
4     explicit dsu(size_t size) : pa(size) { iota(pa.begin(), pa.end(), 0); }
5 };
```

```
1 class Dsu:
2     def __init__(self, size):
3         self.pa = list(range(size))
```

查询

我们需要沿着树向上移动，直至找到根节点。



▼ 实现



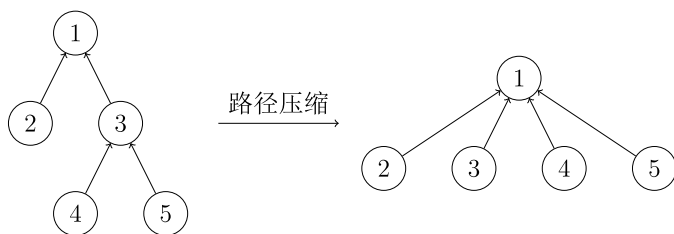
C++Python

```
1 size_t dsu::find(size_t x) { return pa[x] == x ? x : find(pa[x]); }
```

```
1 def find(self, x):
2     return x if self.pa[x] == x else self.find(self.pa[x])
```

路径压缩

查询过程中经过的每个元素都属于该集合，我们可以将其直接连到根节点以加快后续查询。



▼ 实现



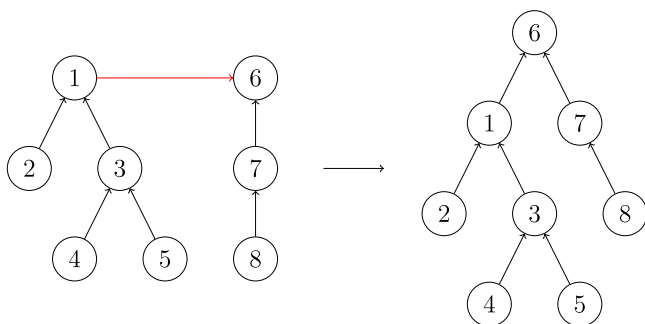
C++Python

```
1 size_t dsu::find(size_t x) { return pa[x] == x ? x : pa[x] = find(pa[x]); }
```

```
1 def find(self, x):
2     if self.pa[x] != x:
3         self.pa[x] = self.find(self.pa[x])
4     return self.pa[x]
```

合并

要合并两棵树，我们只需要将一棵树的根节点连到另一棵树的根节点。



▼ 实现



C++Python

```
1 void dsu::unite(size_t x, size_t y) { pa[find(x)] = find(y); }
```

```
1 def union(self, x, y):  
2     self.pa[self.find(x)] = self.find(y)
```

启发式合并

合并时，选择哪棵树的根节点作为新树的根节点会影响未来操作的复杂度。我们可以将节点较少或深度较小的树连到另一棵，以免发生退化。

► 具体复杂度讨论

按节点数合并的参考实现：

▼ 实现



C++Python

```
1 struct dsu {  
2     vector<size_t> pa, size;  
3  
4     explicit dsu(size_t size_) : pa(size_), size(size_, 1) {  
5         iota(pa.begin(), pa.end(), 0);  
6     }  
7  
8     void unite(size_t x, size_t y) {  
9         x = find(x), y = find(y);  
10        if (x == y) return;  
11        if (size[x] < size[y]) swap(x, y);  
12        pa[y] = x;  
13        size[x] += size[y];  
14    }  
15};
```

```
1 class Dsu:  
2     def __init__(self, size):  
3         self.pa = list(range(size))  
4         self.size = [1] * size  
5  
6     def union(self, x, y):  
7         x, y = self.find(x), self.find(y)  
8         if x == y:  
9             return  
10        if self.size[x] < self.size[y]:  
11            x, y = y, x  
12        self.pa[y] = x  
13        self.size[x] += self.size[y]
```

删除

要删除一个叶子节点，我们可以将其父亲设为自己。为了保证要删除的元素都是叶子，我们可以预先为每个节点制作副本，并将其副本作为父亲。

▼ 实现



C++Python

```

1 struct dsu {
2     vector<size_t> pa, size;
3
4     explicit dsu(size_t size_) : pa(size_ * 2), size(size_ * 2, 1) {
5         iota(pa.begin(), pa.begin() + size_, size_);
6         iota(pa.begin() + size_, pa.end(), size_);
7     }
8
9     void erase(size_t x) {
10         --size[find(x)];
11         pa[x] = x;
12     }
13 };

```

```

1 class Dsu:
2     def __init__(self, size):
3         self.pa = list(range(size, size * 2)) * 2
4         self.size = [1] * size * 2
5
6     def erase(self, x):
7         self.size[self.find(x)] -= 1
8         self.pa[x] = x

```

移动

与删除类似，通过以副本作为父亲，保证要移动的元素都是叶子。

▼ 实现



C++Python

```

1 void dsu::move(size_t x, size_t y) {
2     auto fx = find(x), fy = find(y);
3     if (fx == fy) return;
4     pa[x] = fy;
5     --size[fx], ++size[fy];
6 }

1 def move(self, x, y):
2     fx, fy = self.find(x), self.find(y)
3     if fx == fy:
4         return
5     self.pa[x] = fy
6     self.size[fx] -= 1
7     self.size[fy] += 1

```

复杂度

时间复杂度

同时使用路径压缩和启发式合并之后，并查集的每个操作平均时间仅为 $\alpha(n)$ ，其中 α 为阿克曼函数的反函数，其增长极其缓慢，也就是说其单次操作的平均运行时间可以认为是一个很小的常数。

[Ackermann 函数](#) 的定义是这样的：

而反 Ackermann 函数 的定义是阿克曼函数的反函数，即为最大的整数 m 使得 $A(m, m) \leq n$ 。

时间复杂度的证明 [在这个页面中](#)。

空间复杂度

显然为 。

带权并查集

我们还可以在并查集的边上定义某种权值、以及这种权值在路径压缩时产生的运算，从而解决更多的问题。比如对于经典的「NOI2001」食物链，我们可以在边权上维护模 3 意义下的加法群。

例题

▼ [UVa11987 Almost Union-Find](#)

实现类似并查集的数据结构，支持以下操作：

1. 合并两个元素所属集合
2. 移动单个元素
3. 查询某个元素所属集合的大小及元素和

► [参考代码](#)

习题

[「NOI2015」程序自动分析](#)

[「JSOI2008」星球大战](#)

[「NOI2001」食物链](#)

[「NOI2002」银河英雄传说](#)

其他应用

[最小生成树算法](#) 中的 Kruskal 和 [最近公共祖先](#) 中的 Tarjan 算法是基于并查集的算法。

相关专题见 [并查集应用](#)。

参考资料与拓展阅读

1. [知乎回答：是否在并查集中真的有二分路径压缩优化？](#)
2. Gabow, H. N., & Tarjan, R. E. (1985). A Linear-Time Algorithm for a Special Case of Disjoint Set Union. JOURNAL OF COMPUTER AND SYSTEM SCIENCES, 30, 209-221.[PDF](#)

-
1. Tarjan, R. E., & Van Leeuwen, J. (1984). Worst-case analysis of set union algorithms. Journal of the ACM (JACM), 31(2), 245-281.[ResearchGate PDF](#) ↗
 2. Yao, A. C. (1985). On the expected performance of path compression algorithms.[SIAM Journal on Computing. 14\(1\), 129-133.](#) ↗
-

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[Marcythm](#), [H-J-Granger](#), [HeRaNO](#), [iamtwz](#), [ouuan](#), [sailordiar](#), [Xeonacid](#), [abc1763613206](#), [aofall](#), [CCXXXI](#), [chaiHdu](#), [Chrogeek](#), [ChungZH](#), [CoelacanthusHex](#), [countercurrent-time](#), [Enter-tainer](#), [gi-b716](#), [Haohu Shen](#), [Henry-ZHR](#), [lr1d](#), [JuicyMio](#), [juicymio](#), [ksyx](#), [leoleoasd](#), [mcendu](#), [Menci](#), [mgt](#), [NachtgeistW](#), [orzAtalod](#), [Persdre](#), [shawlleyw](#), [shuzhouliu](#), [SJoshua](#), [sshwy](#), [stevebraveman](#), [StudyingFather](#), [SukkaW](#), [symkub](#), [Tedhjl](#), [Tiphereth-A](#), [YanJun-Zhao](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用